

# Deep Learning for Solving Dynamic Models

## Session 9 — Continuous-time heterogeneous agents: HJB, KFE, and PINNs

Simon Scheidegger

HEC, University of Lausanne

University of Padova

September 24, 2026

Based on: Achdou, Han, Lasry, Lions & Moll (2022), *Rev. Econ. Stud.* 89(1), 45–86.

Gu, Laurière, Merkel & Payne (2024), “Global Solutions to Master Equations.” arXiv:2406.13726.

# Roadmap of This Session (40 min)

1. **Why continuous time** — what the reformulation buys us (5 min)
2. **The HJB–KFE system** — individual optimization plus the cross-sectional distribution; Aiyagari mapped operator by operator (20 min)
3. **Numerics** — upwind finite differences versus a PINN on the identical problem, and when each stops being viable (15 min)

## Where this sits

Session 8 turned a PDE residual into a loss. Here that machinery meets the canonical heterogeneous-agent model, and we compare it against the classical finite-difference benchmark.

# Why Continuous Time?

## Advantages of continuous-time formulation:

- ▶ **Analytical tractability:** Ito calculus provides closed-form FOCs
- ▶ **Clean separation:** HJB (backward)  $\leftrightarrow$  KFE (forward)
- ▶ **Powerful numerical methods:** finite differences, PINNs
- ▶ **No expectations operator:** replaced by differential operators
- ▶ **Connection to mean field games** (Lasry & Lions, 2007)

## Key References

- ▶ Achdou, Han, Lasry, Lions & Moll (2022), *REStud*
- ▶ Kaplan, Moll & Violante (2018), *AER*
- ▶ Moll lecture notes:  
[benjaminmoll.com/lectures/](https://benjaminmoll.com/lectures/)

## Discrete $\rightarrow$ Continuous

Day 4 (DEQNs, Young's method) used *discrete time*.

Today: *continuous time*  $\Rightarrow$  PDEs.

# The Aiyagari–Bewley–Huggett Framework

- ▶ Benchmark models in “quantitative” macroeconomics:
  - ▶ **Bewley** (1980, 1983): precautionary saving with borrowing constraints
  - ▶ **Huggett** (1993, *JEDC*): endowment economy, risk-free rate determination
  - ▶ **Aiyagari** (1994, *QJE*): production economy, general equilibrium
  - ▶ **Krusell & Smith** (1998, *JPE*): adds aggregate TFP shocks
- ▶ Core mechanism: **idiosyncratic labor income risk** is **uninsurable**
  - ▶ Households want to borrow when income is low → borrowing limits prevent this
  - ▶ Instead, households accumulate assets as **precautionary savings**
- ▶ These are building blocks for **HANK** models (Kaplan, Moll & Violante, 2018)
  - ▶ Heterogeneous Agent New Keynesian: monetary/fiscal policy with inequality

# Itô Processes and SDEs

## General Itô Process

A stochastic process  $X_t$  follows an **Itô process** if:

$$dX_t = \mu(X_t, t) dt + \sigma(X_t, t) dB_t$$

- ▶  $\mu(X_t, t)$ : **drift** (deterministic trend)
- ▶  $\sigma(X_t, t)$ : **diffusion coefficient** (volatility / noise intensity)
- ▶  $B_t$ : standard Brownian motion

## Important examples:

### Geometric Brownian Motion (GBM)

$$dS_t = \mu S_t dt + \sigma S_t dB_t$$

Used for: stock prices, GDP

### Ornstein–Uhlenbeck (OU) Process

$$dZ_t = \eta(\bar{Z} - Z_t) dt + \sigma dB_t$$

Used for: productivity, interest rates

# What Is the Kolmogorov Forward Equation?

The **Kolmogorov forward equation** (KFE), also known as the **Fokker–Planck equation**, describes how the **probability density** of a stochastic process evolves over time.

**Origin:** physics and applied mathematics.

- ▶ **A.N. Kolmogorov** (1931): general theory of continuous Markov processes
- ▶ **A.D. Fokker** (1914) & **M. Planck** (1917): diffusion of particles
- ▶ Applications: heat conduction, fluid dynamics, plasma physics, chemical kinetics, population genetics, neuroscience, ...

**Key idea:** micro dynamics (drift + noise)  $\Rightarrow$  macro density law of motion.



A.N. Kolmogorov, Wikimedia Commons

# The KFE: General Itô Diffusion

For  $dX_t = \mu(X_t) dt + \sigma(X_t) dB_t$  with **state-dependent** drift and diffusion, the density  $g(x, t)$  satisfies the **general KFE**:

## Kolmogorov Forward / Fokker–Planck Equation

$$\frac{\partial g}{\partial t}(x, t) = -\frac{\partial}{\partial x} [\mu(x) g(x, t)] + \frac{1}{2} \frac{\partial^2}{\partial x^2} [\sigma(x)^2 g(x, t)]$$

**Important:** when  $\sigma$  depends on  $x$ , it enters *inside* the second derivative:

$$\frac{1}{2} \partial_{xx} [\sigma(x)^2 g] \neq \frac{\sigma(x)^2}{2} \partial_{xx} g$$

In **divergence form**:  $\partial_t g = -\partial_x \underbrace{\left[ \mu(x) g - \frac{1}{2} \partial_x (\sigma^2 g) \right]}_{=: \text{probability flux } J(x, t)}$  Conservation law:  $\partial_t g + \partial_x J = 0$ .

# KFE in Economics: The Wealth Distribution

**Setup:** Agent  $i$  has wealth  $a_t^i$  and income state  $n_t^i \in \{n_1, n_2\}$ .

Individual wealth evolves as:

$$da_t^i = \underbrace{s(a_t^i, n_t^i)}_{\text{savings} = \text{income} - \text{consumption}} dt, \quad a_t^i \geq \underline{a}$$

Income  $n_t^i$  switches between  $n_1 \leftrightarrow n_2$  with Poisson intensities  $\lambda_1, \lambda_2$ .

The **cross-sectional density**  $g_t(a, n)$  satisfies:

$$\partial_t g_t(a, n) = -\partial_a [s(a, n) g_t(a, n)] - \lambda(n) g_t(a, n) + \lambda(\hat{n}) g_t(a, \hat{n})$$

where  $\hat{n}$  denotes the complementary income state.

- ▶  $-\partial_a [s \cdot g]$ : flow of agents along the wealth axis (savings)
- ▶  $-\lambda(n) g(a, n)$ : agents **leaving** state  $n$  (exit from  $n \rightarrow \hat{n}$ )
- ▶  $+\lambda(\hat{n}) g(a, \hat{n})$ : agents **entering** state  $n$  (entry from  $\hat{n} \rightarrow n$ )

# Stationary Wealth Distribution

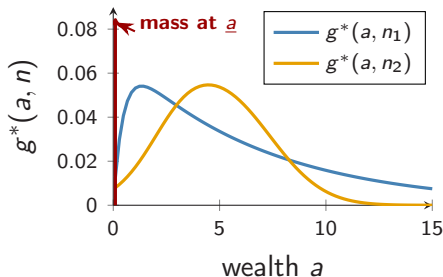
Setting  $\partial_t g = 0$ , the **stationary** (ergodic) distribution solves:

$$0 = -\partial_a [s(a, n) g^*(a, n)] \\ - \lambda(n) g^*(a, n) + \lambda(\hat{n}) g^*(a, \hat{n})$$

Key features of  $g^*(a, n)$ :

- ▶ **Right-skewed**: many poor, few rich
- ▶ **Mass point** at borrowing constraint  $\underline{a}$  for low-income agents
- ▶ Separate densities for each income state
- ▶ Normalization:  $\sum_n \int_{\underline{a}}^{\infty} g^*(a, n) da = 1$

This is the object solved numerically in Aiyagari (1994) and Achdou et al. (2022).



# Deriving the HJB: Five Steps

**Step 1 — Dynamic Programming Principle.** For small  $h > 0$ :

$$V(a, n) = \max_c \left\{ \int_0^h e^{-\rho t} u(c_t) dt + e^{-\rho h} \mathbb{E}[V(a_h, n_h) \mid a_0 = a, n_0 = n] \right\}$$

**Step 2 — Itô's lemma** on  $V(a_t, n_t)$  (between jumps of  $n$ ):

$$dV = V'(a, n) \cdot s(c, a, n) dt$$

(no diffusion term since  $da = s dt$  is deterministic between jumps)

**Step 3 — Take expectations**, account for Poisson jumps:

$$\mathbb{E}[dV] = [V'(a, n) \cdot s + \lambda(n)(V(a, \hat{n}) - V(a, n))] dt$$

**Step 4 — Substitute** into DPP, divide by  $h$ , let  $h \rightarrow 0$ :

**Step 5 — Impose optimality** over  $c$ :

$$\rho V(a, n) = \max_c \{ u(c) + V'(a, n) \cdot (wn + ra - c) + \lambda(n)(V(a, \hat{n}) - V(a, n)) \}$$

# The HJB Equation: Interpretation

$$\rho V(a, n) = \max_c \left\{ u(c) + \underbrace{V'(a, n) \cdot s(a, n, c)}_{\text{marginal value} \times \text{savings}} + \underbrace{\lambda(n)(V(a, \hat{n}) - V(a, n))}_{\text{expected change from income switch}} \right\}$$

**Interpretation** (“required return on the value function”):

- ▶ **LHS:**  $\rho V$  = “required return” = discount rate  $\times$  asset value
- ▶ **RHS:** “dividend” + “capital gain” = total return
  - ▶  $u(c)$ : flow utility (“dividend”)
  - ▶  $V'(a, n) \cdot s$ : expected capital gain from saving
  - ▶  $\lambda(n)(V(\hat{n}) - V(n))$ : expected gain/loss from income switch

The HJB is an **asset pricing equation**:  
the value function is priced like a financial asset.

# Optimal Policy: The Consumption Function

**First-order condition** (differentiate HJB w.r.t.  $c$ ):

$$u'(c^*) = V'(a, n) \quad \Longleftrightarrow \quad c^*(a, n) = (u')^{-1}(V'(a, n))$$

For CRRA utility  $u(c) = \frac{c^{1-\gamma}}{1-\gamma}$  with  $u'(c) = c^{-\gamma}$ :

$$c^*(a, n) = [V'(a, n)]^{-1/\gamma}$$

**Savings function:**

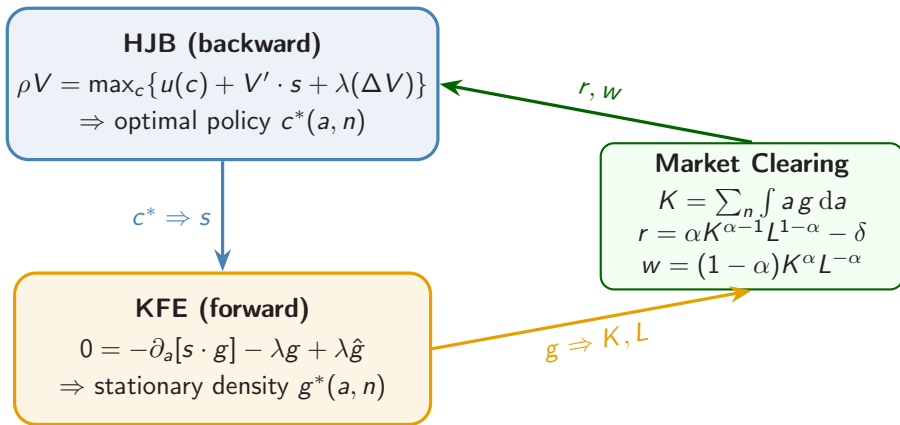
$$s(a, n) = wn + ra - c^*(a, n)$$

**State constraint at  $a = \underline{a}$ :**

$$\begin{aligned} 0 &< c^*(\underline{a}, n) \leq wn + r\underline{a}, \\ c^*(\underline{a}, n) &= \min\{[V_a(\underline{a}, n)]^{-1/\gamma}, wn + r\underline{a}\}, \\ s(\underline{a}, n) &= wn + r\underline{a} - c^*(\underline{a}, n) \geq 0. \end{aligned}$$

Interior:  $u'(c^*) = V_a$ . Boundary: constrain consumption so wealth cannot drift below  $\underline{a}$ .

# The Coupled HJB–KFE System



**Solution method:** iterate until convergence (Aiyagari, no aggregate shocks):

1. Guess  $r \rightarrow$  solve HJB for  $V, c^* \rightarrow$  solve KFE for  $g^* \rightarrow$  compute  $K \rightarrow$  update  $r$

# Aiyagari (1994): Formal Stationary PDE System

**Production side** (no aggregate shocks):

$$Y = K^\alpha L^{1-\alpha}, \quad r = \alpha K^{\alpha-1} L^{1-\alpha} - \delta, \quad w = (1 - \alpha) K^\alpha L^{-\alpha}.$$

**Household problem:**

$$\max_{\{c_t\}} \mathbb{E} \left[ \int_0^\infty e^{-\rho t} u(c_t) dt \right], \quad da_t = (wz_t + ra_t - c_t) dt, \quad a_t \geq \underline{a}, \quad z_t \in \{z_1, z_2\}.$$

**HJB:**

$$\begin{aligned} \rho V(a, z) &= \max_c \{ u(c) + V_a(a, z) [wz + ra - c] + \lambda(z) (V(a, \hat{z}) - V(a, z)) \}, \\ u'(c^*) &= V_a, \quad s(a, z) := wz + ra - c^*(a, z). \end{aligned}$$

**KFE** on  $(\underline{a}, \infty)$ :

$$0 = -\partial_a [s(a, z) g^*(a, z)] - \lambda(z) g^*(a, z) + \lambda(\hat{z}) g^*(a, \hat{z}).$$

**Market clearing & normalization:**

$$K = \sum_z \int_{\underline{a}}^\infty a g^*(a, z) da, \quad L = \sum_z \int_{\underline{a}}^\infty z g^*(a, z) da, \quad \sum_z \int_{\underline{a}}^\infty g^*(a, z) da = 1.$$

Unknowns:  $(V, g^*, r, w, K)$  (or  $(V, g^*, r)$  with  $w = w(r)$ ).

# Huggett vs. Aiyagari: Key Differences

|          | Huggett (1993)            | Aiyagari (1994)                            |
|----------|---------------------------|--------------------------------------------|
| Economy  | Endowment                 | Production ( $Y = K^\alpha L^{1-\alpha}$ ) |
| Asset    | Bond $b$                  | Capital claim $a$                          |
| Supply   | Zero net ( $\int b = 0$ ) | Positive ( $\int a = K > 0$ )              |
| Price    | Bond return $q$           | Interest rate $r$ , wage $w$               |
| Clearing | $\int b g db = 0$         | $\int a g da = K(r)$                       |

## How the distributions differ:

### Huggett:

- ▶ Wealth centered around 0 (zero net supply)
- ▶ Large mass at borrowing constraint
- ▶ Symmetric-ish with constrained left tail

### Aiyagari:

- ▶ Wealth shifted right ( $K > 0$ )
- ▶ Long right tail (wealthy agents)
- ▶ Smaller mass at constraint

# The Master Equation

**Key idea:** Substitute KFE, market clearing, and belief consistency **into** the HJBE. Result: **one PDE** characterizing the equilibrium value function:

## Master Equation

$$\begin{aligned} 0 = & -\rho V(a, n, z, g) + u(c^*) + V_a [w(z, g)n + r(z, g)a - c^*] \\ & + \lambda(n)(V(a, \hat{n}, z, g) - V(a, n, z, g)) + V_z \mu^z(z) + \frac{1}{2}(\sigma^z)^2 V_{zz} \\ & + \sum_{j \in \{1, 2\}} \int_{\underline{a}}^{\infty} \frac{\delta V}{\delta g_j}(a, n, z, g)(y) \mu_j^g(y, z, g) dy =: \mathcal{L}V. \end{aligned}$$

where  $c^*$  satisfies  $u'(c^*) = \partial_a V(a, n, z, g)$ .

**“One equation to rule them all”**

HJB + KFE + market clearing = master equation

# Finite Differences: The Classical Approach

How have economists traditionally solved these models?

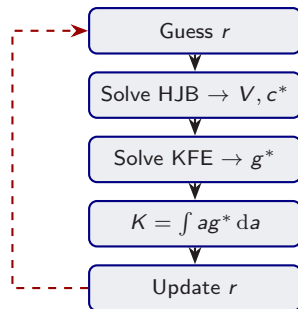
**Finite difference method** (Achdou et al., 2022):

1. Discretize wealth  $a$  on a grid  $\{a_1, \dots, a_I\}$
2. Approximate derivatives by finite differences:

$$V'(a_i) \approx \frac{V_{i+1} - V_i}{\Delta a} \quad \text{or} \quad \frac{V_i - V_{i-1}}{\Delta a}$$

3. **Upwind scheme**: use forward diff. when savings  $> 0$ , backward when  $< 0$
4. Iterate HJB  $\rightarrow$  policy  $\rightarrow$  KFE  $\rightarrow$  aggregates  $\rightarrow$  prices  $\rightarrow$  HJB

This summarizes the finite-difference workflow; code examples are provided later in Python.



# Finite Differences: What They Produce

## Finite-Difference Example: Aiyagari GE

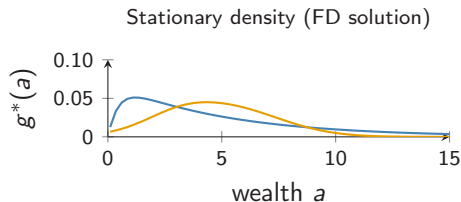
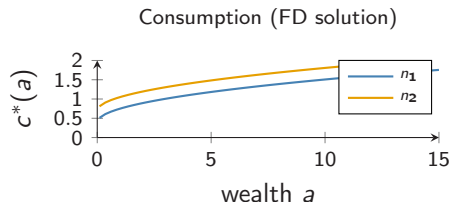
The classical FD approach solves:

- ▶ HJB block: upwind discretization + implicit updates
- ▶ KFE block: transition operator + stationary distribution
- ▶ Price block: root-finding for market-clearing  $r$

Produces:

- ▶ Value function  $V(a, n)$
- ▶ Consumption/savings policy  $c^*(a, n)$
- ▶ Stationary density  $g^*(a, n)$
- ▶ Equilibrium prices  $r^*, w^*$

This is our ground truth for validating deep learning methods.



# Why Finite Differences Are Not Enough

## FD works well for:

- ✓ Stationary HJB–KFE systems
- ✓ Low-dimensional states  $(a, n)$
- ✓ Monotone convergence theory
- ✓ Fast sparse linear algebra

## FD is strained by:

- ✗ Global KS master equation
- ✗ State  $(a, n, z, g)$  with distribution  $g$
- ✗ Dense tensor-product grids
- ✗ Moment truncation of  $g$

## Takeaway

Neural methods do not remove the equilibrium conditions; they approximate value and distribution maps without building a full grid over  $g$ .

# From Single PDE to Coupled System

**Block 1 (PINNs):** Solve a *single* PDE (Poisson, HJB cake-eating, Black–Scholes).

**Block 2 (this block):** Solve a *coupled system* of PDEs arising from equilibrium:

| Component       | PDE                                                         | Unknown           |
|-----------------|-------------------------------------------------------------|-------------------|
| HJB (backward)  | $\rho V = \max_c \{u(c) + V' \cdot s + \lambda(\Delta V)\}$ | Value $V(a, n)$   |
| KFE (forward)   | $0 = -\partial_a[s \cdot g] - \lambda g + \lambda \hat{g}$  | Density $g(a, n)$ |
| Market clearing | $K = \sum_n \int a g \, da$                                 | Prices $r, w$     |

**The coupling:** HJB output ( $c^* \rightarrow s$ ) is the KFE input (drift).

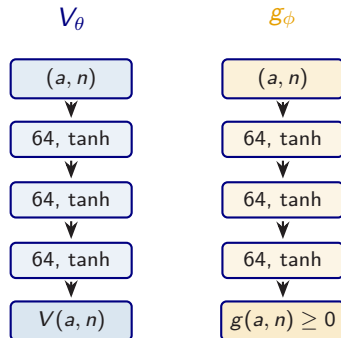
KFE output ( $g \rightarrow K, L$ ) determines prices ( $r, w$ ) that enter HJB.

**PINN approach:** Parameterize *both*  $V$  and  $g$  by neural networks.  
Minimize *all* residuals **jointly** via a single loss function.

# PINN Architecture for Aiyagari/Huggett

## Two neural networks:

- ▶  $V_\theta(a, n)$ : value function network
  - ▶ Input:  $(a, n)$  ( $n$  encoded as 0/1)
  - ▶ Output: scalar  $V$
  - ▶ Architecture: 3–4 hidden layers, 64 neurons, tanh
- ▶  $g_\phi(a, n)$ : density network
  - ▶ Input:  $(a, n)$
  - ▶ Output: scalar  $g \geq 0$  (use softplus)
  - ▶ Same architecture



## Derived quantities (via auto-diff):

- ▶  $V'_\theta = \partial_a V_\theta \Rightarrow$  consumption:  $c^* = (V'_\theta)^{-1/\gamma}$
- ▶  $\partial_a g_\phi \Rightarrow$  KFE residual

# Training Algorithm: Jointly Solving HJB + KFE

## PINN Training (Stationary Equilibrium)

- 1: Initialize  $V_\theta$ ,  $g_\phi$ , guess  $r^{(0)}$  (*FD reserved for validation*)
- 2: **for**  $k = 1, \dots, N_{\text{epochs}}$  **do**
- 3:   Sample  $M$  collocation points  $\{(a_m, n_m)\}$
- 4:   Compute  $c_m^* = (V'_\theta(a_m))^{-1/\gamma}$
- 5:   Compute  $R_{\text{HJB}}^m, R_{\text{KFE}}^m$  via auto-diff
- 6:   Integrate KFE on a fixed asset grid
- 7:   Compute  $\mathcal{L}_{\text{SC}}, \mathcal{L}_{\text{flux}}, \mathcal{L}_{\text{mass}}, \mathcal{L}_{\text{MC}}$
- 8:   Total loss:  $\mathcal{L} = \sum \kappa_i \mathcal{L}_i$ ; update  $\theta, \phi$  (Adam)
- 9:   **if**  $k \bmod N_r = 0$  **then**
- 10:      $r \leftarrow r + \eta(K^s - K^d)$
- 11:   **end if**
- 12: **end for**
- 13: Freeze collocation grid; run FP64 L-BFGS polish

### Key choices:

- **Collocation:** sample  $(a, n)$  uniformly on  $[\underline{a}, \bar{a}] \times \{n_1, n_2\}$
- **Price update:** update  $r$  slowly (every  $N_r$  steps) to let NN adapt
- **Loss weights:** start with  $\kappa_{\text{HJB}} = \kappa_{\text{KFE}} = 1$ , larger  $\kappa_{\text{SC}}, \kappa_{\text{flux}}, \kappa_{\text{MC}}$
- **Optimizer:** Adam,  $\text{lr} \sim 10^{-3}$ , then FP64 L-BFGS
- **Validation:** compare against FD only after training

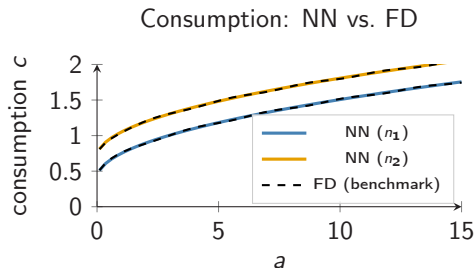
Same philosophy as Day 5: **economic equations**  
= **loss function**; **auto-diff** = **all derivatives**.

# Results: Aiyagari Model (No Aggregate Shocks)

**Validation:** Compare EMINN solutions against finite-difference benchmark.

| Method            | ME Loss              | MSE(NN,FD)           |
|-------------------|----------------------|----------------------|
| Finite Pop. NN    | $3.1 \times 10^{-5}$ | $4.8 \times 10^{-5}$ |
| Discrete State NN | $9.3 \times 10^{-6}$ | $6.6 \times 10^{-5}$ |

- ▶ ME residual:  $\sim 10^{-5}$  (excellent)
- ▶ Agreement with FD:  $\sim 10^{-5}$  MSE
- ▶ Both approaches match FD closely
- ▶ Consistent across multiple runs



# Why PINNs over Finite Differences?

For stationary problems (Huggett/Aiyagari without aggregate shocks):

|                         | Finite Differences    | PINN                    |
|-------------------------|-----------------------|-------------------------|
| Grid required           | Yes ( $I$ points)     | <b>No</b> (collocation) |
| Upwind scheme           | Manual implementation | Auto-diff handles it    |
| Curse of dimensionality | $O(I^d)$              | $O(M \cdot  \theta )$   |
| Extension to higher dim | Very hard             | Natural                 |
| Borrowing constraint    | Special treatment     | Soft penalty / hard BC  |
| Accuracy (2D)           | $\sim 10^{-6}$        | $\sim 10^{-4}$          |
| Speed (2D)              | Faster                | Slower                  |

**Verdict:** *FD is faster and more accurate* for low-dim stationary problems — use it as the gold-standard benchmark. PINNs **scale to high dimensions**: aggregate shocks (KS, state  $\ni g$ ), multiple assets, richer heterogeneity.

$\Rightarrow$  Part III: **EMINNs** for the master equation.

# Method Comparison: FD vs. PINN vs. EMINN

|                          | Finite Diff.      | PINN              | EMINN                   |
|--------------------------|-------------------|-------------------|-------------------------|
| Global master eq.        | ✗                 | ✗                 | ✓                       |
| Stationary equilibrium   | ✓                 | ✓                 | ✓                       |
| Grid required            | Yes               | No                | No                      |
| High-dimensional scaling | Limited           | Better, validate  | Better, validate        |
| Handles $g$ as state     | Standard: no      | No                | Via $\hat{\phi}$        |
| Theory                   | Strong in low dim | Limited           | Limited                 |
| Low-dim accuracy         | $\sim 10^{-6}$    | problem-dependent | reported $\sim 10^{-5}$ |

## Bottom line:

- ▶ **Stationary, low-dim:** FD is fast and reliable. Use it as benchmark.
- ▶ **Aggregate shocks:** EMINNs target global master-equation solutions.
- ▶ **PINNs:** useful bridge for the coupled HJB–KFE system.

# Code Companion: Run This Live

## Day 6 companion notebooks (self-contained, PyTorch):

- ▶ **NB 06** (*self-study*): 06\_PE\_Discrete\_HJB\_PINN.ipynb — 1D PE with discrete income: upwind FD + PINN
- ▶ **NB 07** (*self-study*): 07\_PE\_Diffusion\_HJB\_PINN.ipynb — 2D PE with OU earnings: 2D FD + PINN
- ▶ **NB 08** (*in-class*): 08\_Aiyagari\_Continuous\_Time\_FD\_and\_PINN\_PyTorch.ipynb — full GE Aiyagari: HJB + KFE + market clearing

## Suggested in-class demo flow:

1. Start with NB 06 (simplest, self-study): FD convergence, then PINN overlay
2. Move to NB 07 (self-study): 2D contour plots, cross-sections, mesh-free advantage
3. Focus on NB 08 (in-class): full GE loop, PINN vs FD agreement diagnostics